

**72.39 - Autómatas, Teoría de Lenguajes y Compiladores**

# **Stage I: Diseño**



Asin, Gerónimo - 65486

Garbarz, Andrés Matías - 65488

Lee, Sebastián Nicolás - 64675

Lifischtz, Santiago - 65782

**Fecha de entrega:** 05/04/2026

# **Índice**

|                              |    |
|------------------------------|----|
| 1. Introducción.....         | 3  |
| 2. Repositorio.....          | 3  |
| 3. Dominio.....              | 3  |
| 4. Funcionalidades.....      | 4  |
| 5. Casos de prueba.....      | 14 |
| 6. Ejemplos de sintaxis..... | 19 |
| 7. Bibliografía.....         | 20 |

# **1. Introducción**

Este trabajo presenta el diseño de un lenguaje específico orientado a la generación de música en formato MIDI. El objetivo principal es ofrecer una herramienta declarativa que permita describir diferentes composiciones musicales de manera expresiva y reutilizable. El uso de este lenguaje, también brinda al usuario una abstracción de los detalles técnicos de la generación de archivos MIDI manteniendo un alto control sobre lo que se genera.

En este informe, se introduce el diseño propuesto para la etapa de Stage I: Diseño, en donde se especifica el dominio del lenguaje junto con sus funcionalidades y ejemplos de uso.

## **2. Repositorio**

<https://github.com/SantiagoLifischtz/TLA-20261Q>

## **3. Dominio**

El lenguaje propuesto permite escribir música de manera declarativa, produciendo un archivo MIDI como salida. Su sintaxis busca reducir el esfuerzo involucrado en describir una composición musical, ofreciendo herramientas para transponer, repetir y estructurar secuencias de notas de forma reutilizable.

En música, una escala es un conjunto ordenado de notas que define la “tonalidad” de una pieza: cuáles notas suenan bien juntas y cuál es la nota central. Dentro de una escala, cada nota ocupa una posición numerada llamada grado. Por ejemplo, en la escala de Do mayor, el primer grado es Do, el tercero es Mi y el quinto es Sol. Escribir notas como grados en lugar de nombrarlas directamente tiene una ventaja clave: si se decide cambiar la escala, todas las notas se transponen automáticamente sin tener que modificarlas una por una.

El usuario puede definir patrones con nombre y reutilizarlos a lo largo de la composición, evitando repetir las mismas secuencias de notas. Un patrón es simplemente una secuencia de eventos musicales que puede estar compuesta por otros patrones. El lenguaje también incluye generadores: construcciones predefinidas que producen automáticamente ciertos tipos de patrones, como arpeggios o strums.

## 4. Funcionalidades

### Expresiones numéricas

Para cualquier valor numérico en un programa, se puede utilizar una expresión que se resuelve en tiempo de compilación.

- Un número por sí solo se considera un entero.
- Si se incluye un punto decimal, se considera de punto flotante.
- Se pueden utilizar los siguientes operadores:

|    |                                                                                                  |
|----|--------------------------------------------------------------------------------------------------|
| +  | Suma                                                                                             |
| -  | Resta truncada (resultados negativos se consideran 0)                                            |
| *  | Producto                                                                                         |
| /  | División (siempre resulta en punto flotante, el programa no compila si el operando derecho es 0) |
| () | Paréntesis (agrupación)                                                                          |

### Tiempos

El lenguaje provee palabras clave que representan duraciones musicales estándar. Internamente cada una equivale a un número: la unidad base (1) representa una figura entera, y las demás son fracciones de esa unidad. También tienen versiones abreviadas (aquí escritas a la derecha de los ‘|’)

- whole | w → 1
- half | h → 0.5
- quarter | q → 0.25
- eighth | e → 0.125
- sixteenth | s → 0.0625
- thirtysecond | ths → 0.03125
- sixtyfourth | sxf → 0.015625

Hay una excepción notable, la palabra `dot`, que se traduce como una multiplicación por 1.5. Esta es la funcionalidad de una “nota punteada” en una partitura, incrementando su duración un 50%.

- `dot` → \*1.5

### Notas

La primitiva principal del lenguaje. Define el comienzo de una nota junto con su duración. El tono se define mediante la notación americana: la letra de esa nota (C, D, E, F, G, A, B), opcionalmente seguida de ‘#’ para subir medio tono o ‘b’ para bajarlo, y el número de octava (registro de altura). La duración es una expresión numérica.

En este ejemplo se muestran múltiples formas de escribir la duración.

```
F#3 eighth;  
A3 e;  
F#4 quarter dot;  
C#4 1/8;  
E4 1/4;  
D4 q * 1.5;
```

## Patrón

Define una secuencia de sonidos, formada por otros patrones. Una nota sola es considerada un patrón. Tiene una duración, calculada en base a las duraciones de sus contenidos.

- **Declaración**

Se declaran con la palabra clave `pattern`, seguida de su nombre y luego su contenido entre llaves.

```
pattern melody_start {  
    F#3 e;  
    A3 e;  
    F#4 q dot;  
    C#4 e;  
    E4 q;  
    D4 q dot;  
    Bb3 e;  
    C#4 q;  
    B3 q;  
    D4 q;  
    E3 h;  
}
```

- **Utilización**

Una vez definido un patrón, se puede escribir su nombre de nuevo para utilizarlo. Esto tiene el efecto equivalente a reescribir su contenido en el nuevo destino.

```
melody_start;  
melody_end;
```

- **Forma inline**

Dentro de un patrón, un bloque de código encerrado con llaves es considerado un patrón. Esto es importante para otras funcionalidades descritas más adelante.

```
F#3 e;  
A3 e;  
F#4 q dot;
```

```

{
    C#4 e;
    E4 q;
    D4 q dot;
}
Bb3 e;
C#4 q;
B3 q;
D4 q;
E3 h;

```

## Espera

Al final de un patrón, se puede colocar opcionalmente un parámetro de espera, siendo el tiempo exacto entre el comienzo de ese patrón y el comienzo del próximo.

Por defecto, la espera es igual a la duración del patrón, lo que llamamos espera implícita: el siguiente patrón comienza exactamente cuando termina el anterior. A dos patrones consecutivos en esta condición los llamamos “ligados”.

En este ejemplo, el patrón `counter_melody` comienza justo cuando suena el segundo `C#4` de `melody_start` (del ejemplo anterior).

```

melody_start 6*quarter;
counter_melody;

```

La espera del último elemento dentro de un patrón contribuye a la duración total de ese patrón. Si el patrón padre tiene espera explícita, la espera del último elemento interno es ignorada, ya que es la espera del padre la que determina cuándo comienza el siguiente patrón externo.

```

{
    C4 q;
    D4 q h;
}
E4 q; // Comienza exactamente dos tiempos después de que
comienza el D4

```

## Acorde

El mecanismo para hacer sonar múltiples patrones en simultáneo. Se denotan como un arreglo de patrones, entre corchetes, separados por comas. Los patrones dentro de un acorde no pueden tener espera explícita.

Un acorde se considera un patrón. Su duración es igual a la máxima duración de los patrones que lo componen. Por ser un patrón, su espera puede explicitar al final; omitirla equivale a que la espera sea igual a su duración.

```
[D2 whole, F#2 whole, A2 whole]; // Re mayor
```

Alternativamente, para evitar repetir tiempos en acordes compuestos solo por notas, se puede escribir un acorde como tonos sin duración, y una única duración al final.

```
[D2, F#2, A2] whole; // Equivalente al ejemplo anterior
```

A pesar de llamarse acorde, es un mecanismo genérico para comenzar varios patrones al mismo tiempo, que podrían ser, por ejemplo, la mano izquierda y derecha de una pieza de piano.

## Silencio

Para introducir tiempo de silencio entre el final de un patrón y el comienzo del siguiente, se usa la palabra clave `rest`. Un silencio tiene duración 0; el tiempo de pausa se especifica mediante su espera.

```
C3 q;  
rest q;  
G3 q;  
rest q;  
E3 q;  
rest h;  
C4 q;
```

## Repetición

El mecanismo para hacer ciclos es la palabra clave `repeat`, seguida de un número entero, y luego un patrón. Es equivalente a escribir el patrón tantas veces como diga el número.

```
repeat 4 {  
    C#2 e;  
    G#2 e;  
    C#3 q;  
}
```

## Step

El concepto de `step` permite definir una unidad temporal base para evitar la repetición explícita de esperas entre notas. Un `step` establece un intervalo de tiempo fijo entre los comienzos de los elementos que contiene.

```
step <duración> {  
    F#3 eighth;  
    A3 e;  
    ...  
}
```

Cada elemento dentro del bloque comienza separado por la duración especificada en el `step`. La duración propia de cada patrón no se ve afectada. El `step` únicamente controla el **desplazamiento temporal entre inicios**, no la duración de las notas. Los patrones directamente dentro de un bloque `step` no pueden tener parámetro de espera.

## Pista

- **Track**

La palabra `track` representa una secuencia musical independiente dentro del archivo MIDI generado. Cada `track` se traduce en un canal MIDI separado.

```
track piano {  
    instrument "Acoustic Grand";  
    [C4, E3, G5] 1/2;  
    // ...  
}
```

Los patrones dentro de una pista se ejecutan de manera secuencial, respetando las reglas de espera. Múltiples pistas se reproducen en paralelo, lo que permite modelar distintas voces o instrumentos dentro de una misma composición.

- **Instrument**

Cada pista debe asociarse a un instrumento MIDI (`instrument`) específico.

```
instrument nombre_instrumento;
```

El nombre del instrumento corresponde a uno de los 128 instrumentos definidos por el estándar **General MIDI** (0 a 127). Afecta únicamente a la pista en la que se declara y genera un evento MIDI de cambio de instrumento (*Program Change*).

Ejemplo de uso:

```
track piano {  
    instrument "Acoustic Grand";
```



```
melody_start;

repeat 2 {
    [C4, E3, G5] 1/2;
}

}
```

A diferencia de un patrón, una pista no puede utilizarse dentro de otra pista ni patrón. Solo puede utilizarse para definir la salida del programa.

## Tempo

La palabra `tempo` define la velocidad de reproducción de la pieza musical.

```
tempo <valor>;
```

El valor se expresa en **beats per minute (BPM)**. Se puede definir por fuera de cualquier bloque, estableciendo el valor por defecto para toda la composición. Puede redefinirse en cualquier punto del programa, generando eventos de cambio de tempo en el archivo MIDI

```
pattern example {
    tempo 120;
    melody_start;
    tempo 90;
    melody_end;
} // Ambos cambios son locales a example
```

```
tempo 140;

track piano {
    instrument "Acoustic Grand";

    melody_start;
    // Por defecto estaría en 140 BPM,
    // a menos que melody_start lo cambie localmente
}
```

## Escala

Similarmente al tempo, se puede definir la escala en la que se encuentra la pieza musical con la palabra clave `key`, seguida de la letra de un tono y opcionalmente un modo (por defecto siendo `major`). Igual que el tempo, si es definida por fuera de todo patrón, su alcance es global, si no es local al patrón donde aparece. Los modos disponibles son los siguientes:

- `ionian` | `major`
- `dorian`

- phrygian
- lydian
- mixolydian
- aeolian | minor
- locrian

```
key F major;
```

Definir una escala **no afecta a las notas absolutas**, sino que se utiliza otro mecanismo para definir notas que sí dependen de la escala, con la palabra clave `degree`, seguida de un identificador de grado y una octava como parámetros.

El identificador es un número entero entre 1 y 7 inclusive, que opcionalmente se le puede agregar '#', 'b' o 'n' para subir o bajar un semitono, o forzar la nota a ser natural.

La octava es un entero no negativo, y se calcula en base a la escala. Es decir, `degree(7,1)` y `degree(1,2)` siempre son notas consecutivas en cualquier escala.

```
key F major;
degree(1, 3) q; // Corresponde a Fa natural
degree(4, 3) q; // El cuarto grado de Fa mayor es Si bemol

// Patrón equivalente:
F3 q;
Bb3 q;
```

## Play

Todo programa requiere un bloque `play` que describe su salida. Puede contener una o más pistas definidas anteriormente, o alternatively, utilizar la palabra `all` para usar todas las que haya.

```
track piano {
    instrument "Acoustic Grand";
    // ...
}

track violin {
    instrument "Violin";
    // ...
}

play {
    piano, violin;
}
```

Todas las pistas en el bloque play inician al mismo tiempo, en paralelo. Si el usuario quisiera hacer una composición donde una pista empieza más tarde, lo puede hacer agregando un silencio al inicio de dicha pista.

## Generadores

Los generadores son construcciones de alto nivel que producen patrones automáticamente a partir de parámetros, devolviendo un patrón y facilitando la generación de estructuras musicales repetitivas o complejas.

- **Strum**

El generador `strum` produce un acorde cuyas notas se ejecutan de manera escalonada en el tiempo.

```
strum(nota1, nota2, ..., notan, duración_total, intervalo)
```

Cada nota comienza separada por el `intervalo`. La secuencia completa se ajusta a la duración total especificada. No compila si con  $n$  notas,  $\text{intervalo} * n > \text{duración}$

- **Arpeggio**

El generador `arpeggio` genera una secuencia ordenada de notas a partir de un conjunto dado.

```
arpeggio(nota1, nota2, ..., notan, duración_total)
```

La duración total se divide equitativamente entre las notas. Las notas se ejecutan secuencialmente.

Ejemplo de uso:

```
pattern example {  
    strum(C4, E4, G4, w, e);  
    arpeggio(C4, E4, G4, h);  
}
```

## Percusión

En el estándar General MIDI, el canal 10 está reservado exclusivamente para percusión. A diferencia de los demás instrumentos, cada nota en este canal no representa un tono sino un sonido percusivo distinto.

Un track se define como percusivo declarando `instrument "Drums"`. Dentro de él, en lugar de notas absolutas, se usan identificadores predefinidos que corresponden a cada sonido del mapa de percusión GM. La duración funciona igual que en cualquier otro patrón.

```
track bateria {
    instrument "Drums";
    electric_bass_drum q;
    closed_hi_hat e;
    acoustic_snare q;
}
```

Los identificadores disponibles son:

| Identificador      | MIDI |
|--------------------|------|
| acoustic_bass_drum | 35   |
| electric_bass_drum | 36   |
| side_stick         | 37   |
| acoustic_snare     | 38   |
| hand_clap          | 39   |
| electric_snare     | 40   |
| low_floor_tom      | 41   |
| closed_hi_hat      | 42   |
| high_floor_tom     | 43   |
| pedal_hi_hat       | 44   |
| low_tom            | 45   |
| open_hi_hat        | 46   |
| low_mid_tom        | 47   |
| hi_mid_tom         | 48   |
| crash_cymbal_1     | 49   |
| high_tom           | 50   |
| ride_cymbal_1      | 51   |
| chinese_cymbal     | 52   |
| ride_bell          | 53   |
| tambourine         | 54   |

| Identificador  | MIDI |
|----------------|------|
| splash_cymbal  | 55   |
| cowbell        | 56   |
| crash_cymbal_2 | 57   |
| vibraslap      | 58   |
| ride_cymbal_2  | 59   |
| hi_bongo       | 60   |
| low_bongo      | 61   |
| mute_hi_conga  | 62   |
| open_hi_conga  | 63   |
| short_whistle  | 71   |
| long_whistle   | 72   |
| short_guiro    | 73   |
| long_guiro     | 74   |
| claves         | 75   |
| hi_wood_block  | 76   |
| low_wood_block | 77   |
| mute_cuica     | 78   |
| open_cuica     | 79   |
| mute_triangle  | 80   |
| open_triangle  | 81   |

## Duración de un patrón

Cuando la espera de un patrón se deja implícita, hay un cálculo interno para determinar su duración. Estas son las traducciones:

- Nota / silencio → duración explícita
- Acorde → duración del patrón más largo
- Patrón → suma de las esperas de los patrones contenidos

```
pattern example1 {  
    C3 w;  
    A2 w;  
}  
// Duración de example1 = 2 (w + w)  
  
pattern example2 {  
    C3 w;  
    A2 w q;  
}  
// Duración de example2 = 1.25 (w + q)
```

## 5. Casos de Prueba

A continuación se presentan X casos de aceptación y Y de rechazo:

- **Casos de aceptación:**

```
// Estructura del programa  
  
tempo 120;  
key C major;  
  
track t1 {...}  
track t2 {...}  
play { t1, t2 }
```

```
// Acepta: Espacios inconsistentes (whitespace-insensitive)  
  
tempo 120;C4 q; E4 e;
```

```
// Acepta: Nota absoluta con duración  
track piano {  
    instrument "Acoustic Grand";
```

```
C4 quarter;  
}
```

```
// Acepta: Declaración y uso de patrón  
pattern motif {  
    E4 e;  
    G4 e;  
    A4 q;  
}  
track piano {  
    instrument "Violin";  
    motif;  
    motif;  
}
```

```
// Acepta: Acorde con duración compartida  
track piano {  
    instrument "Acoustic Grand";  
    [C4, E4, G4] half;  
}
```

```
// Acepta: Saltos de línea en declaraciones  
[  
    C4,  
    E4,  
    G4  
] q;
```

```
// Acepta: Repetición de patrón  
track piano {  
    instrument "Acoustic Grand";  
    repeat 4 {  
        C4 e;  
        E4 e;  
        G4 q;  
    }  
}
```

```
// Acepta: Silencio entre notas  
track piano {  
    instrument "Acoustic Grand";  
    C4 q;  
    rest q;  
    G4 q;  
    rest q;  
}
```

```
}
```

```
// Acepta: Generador strum
track guitar {
    instrument "Acoustic Guitar (nylon)";
    strum(C3, E3, G3, half, e);
}
```

```
// Acepta: Generador arpeggio
track piano {
    instrument "Electric Piano 1";
    arpeggio(C4, E4, G4, B4, whole);
}
```

```
// Acepta: Escala y notas por grado
key C major;
track piano {
    instrument "Acoustic Grand";
    degree 1 q;
    degree 3 q;
    degree 5 h;
}
```

```
// Acepta: Cambio de tempo/key local en un pattern
tempo 120;
key C major;

pattern puente {
    degree 1 q;           // 120 BPM
    tempo 90;             // override local desde acá en adelante
    degree 5 q;           // 90 BPM
    degree 3 h;           // 90 BPM
}                          // al salir, vuelve a 120 BPM
```

```
// acepta: Espera explícita con expresión numérica
pattern intro {
    F#3 e;
    A3 e;
    D4 q dot;
}
track piano {
    instrument "Acoustic Grand";
    intro 3*quarter;
    intro;
}
```

```
// Acepta: Uso combinado de strum y degree
tempo 110;
key G major;

track guitar {
    instrument "Acoustic Guitar";
    strum( degree 3, degree 5, degree 5, degree 3, 1, 1/8);
    strum( degree 3, degree 2, degree 5, degree 3, 1, 1/8);
}
```

- **Casos de rechazo:**

```
// Rechaza: Sintaxis incorrecta (missing ;)
track piano {
    tempo 120;
    C4 q
}
```

```
// Rechaza: Nota sin duración
track piano {
    C4;
}
```

```
// Rechaza: Sintaxis incorrecta (missing closing })
tempo 120;
pattern motif {
    C4 q;
    D4 q;
    E4 q;
    F4 q;

track voicel {
    instrument "Violin";
    motif;
}
```

```
// Rechaza: Campos faltantes en Generadores
tempo 120;
arpeggio(degree 1, degree 3, degree 5); // falta la duración
```

```
// Rechaza: Estructura del programa incompleta (falta tempo)
key C major;
```



```
track t1 {...}  
track t2 {...}  
play { t1, t2 }
```

```
// Rechaza: Acorde con espera en un elemento interno  
[  
  C4 q,  
  E4 q h,  
  G4 q  
];
```

## 6. Ejemplos de sintaxis

```
// Ejemplo: Beethoven - Sinfonía N°5 (motif principal)
tempo 108;

pattern motif {
    G4 e;
    G4 e;
    G4 e;
    Eb4 h dot;
    F4 e;
    F4 e;
    F4 e;
    D4 h dot;
}

track orchestra {
    instrument "Strings Ensemble 1";
    repeat 2 {
        motif;
    }
}

play {
    orchestra;
}
```

```
// Ejemplo: Bach - Minueto en Sol (BWV Anh. 114)
// Cambiar key G major por cualquier otra transpone toda la
pieza
tempo 126;
key G major;

pattern verse_a {
    degree(1, 4) q;
    degree(5, 4) e;
    degree(6, 4) e;
    degree(5, 4) e;
    degree(4, 4) e;
    degree(3, 4) q;
    degree(1, 4) q;
    degree(3, 4) q;
    degree(2, 4) e;
    degree(3, 4) e;
}
```

```

    degree(2, 4) e;
    degree(1, 4) e;
    degree(7, 3) h;
    rest q;
}

track harpsichord {
    instrument "Harpsichord";
    verse_a;
}

play {
    harpsichord;
}

```

```

// Ejemplo: White Stripes - Seven Nation Army (intro)
tempo 118;

pattern riff {
    E2 e;
    rest q;
    E2 e;
    G2 e;
    rest s;
    E2 s;
    rest e;
    D2 e;
    C2 h;
    B1 h;
}

pattern rhythm {
    electric_bass_drum q;
    closed_hi_hat e;
    closed_hi_hat e;
    acoustic_snare q;
    closed_hi_hat e;
    closed_hi_hat e;
}

track bass {

```

```
    instrument "Electric Bass (finger)";
    repeat 4 {
        riff;
    }
}

track drums {
    instrument "Drums";
    repeat 8 {
        rhythm;
    }
}

play {
    bass, drums;
}
```

## **7. Bibliografía**

1. EarMaster. (s.f.). General MIDI instrument list. EarMaster. <https://www.earmaster.com/wiki/music-technology/list-of-general-midi-instruments.html>
2. Wikipedia. (s.f.). Mode (music): Modes and scales. Wikipedia. [https://en.wikipedia.org/wiki/Mode\\_\(music\)](https://en.wikipedia.org/wiki/Mode_(music))
3. Wikipedia. (s.f.). General MIDI. Wikipedia. [https://en.wikipedia.org/wiki/General\\_MIDI](https://en.wikipedia.org/wiki/General_MIDI)